# CH-9 PART - 1

## In-Depth Study Notes: Oracle SQL DML Operations and Transaction Control

As an expert Oracle SQL Developer, understanding how to manipulate data and manage database memory is just as critical as querying it. Based on the execution logs provided (CH9-1.TXT and CH9-2.TXT), these notes detail the architecture of **Transaction Control Language (TCL)** and **Data Manipulation Language (DML)**, specifically focusing on **INSERT**, **UPDATE**, and **DELETE** statements.

## Concept 1: Transaction Control Language (TCL)

**Core Purpose:**Oracle does not automatically save data modifications to the hard disk immediately. Instead, DML changes are temporarily held in the server's RAM (Buffer Cache). TCL commands allow developers to permanently save these memory states to the database tables or completely undo them if an error occurs 1-3.

**Keyword Breakdown:**

- **COMMIT**: Takes all open, uncommitted changes currently sitting in RAM and permanently writes them to the physical database tables 3. Once committed, the transaction ends, and the data is visible to all other users 3.
- **ROLLBACK**: Completely flushes all open changes from the RAM, returning the database to its last committed state 2. It acts as a permanent undo 2.
- **SAVEPOINT**: A memory marker. It breaks a massive transaction into smaller, manageable parts, allowing developers to partially undo a transaction without losing all their work 1, 4.

**Comparison of State Management:**

Command,Action on RAM (Buffer Cache),Action on Disk (Table),Transaction State
COMMIT,Clears the memory buffer.,Permanently saves the data.,Ends the transaction 3.
ROLLBACK,Erases the uncommitted data.,No changes made.,Ends the transaction 2.
ROLLBACK TO SAVEPOINT,Erases data back to the marker.,No changes made.,Keeps transaction open 5.

**Syntax Breakdown:**

```
-- Standard TCL Commands
COMMIT;
ROLLBACK;
SAVEPOINT savepoint_name;
ROLLBACK TO savepoint_name;
```

**Detailed Example:**An application inserts new global regions into the system. It successfully adds 'AFRICA', creates a savepoint, and then attempts to add 'ANTARCTICA' but makes a typo. The system rolls back the typo without losing the successful 'AFRICA' insertion.

```
INSERT INTO R1 VALUES (3,'AFRICA');
SAVEPOINT A;
```

```
INSERT INTO R1 VALUES (4,'ANT');

-- Undo changes performed AFTER savepoint 'A', keep changes BEFORE it
ROLLBACK TO A;

-- Permanently save 'AFRICA'
COMMIT;
```

**Execution Logic:**A **Session** is the total time between a user's login and logout 1. A **Transaction** starts the exact millisecond the very first DML statement (INSERT/UPDATE/DELETE) is executed 1. The Oracle Engine tracks these changes in the Undo Tablespace and Redo Log Buffer. The transaction strictly ends only when a COMMIT or ROLLBACK is issued 1.

**Pro-Tips/Best Practices:**

- **Invalid Savepoints:** If you issue a global COMMIT, all SAVEPOINT markers are immediately destroyed 5. Attempting to ROLLBACK TO A after a COMMIT throws an **ORA-01086: savepoint 'A' never established** error 6.

## Concept 2: Adding Data (INSERT)

**Core Purpose:**The INSERT statement is a DML command used to push brand-new rows into an existing table structure 1, 7.

**Keyword Breakdown:**

- **INSERT INTO**: The base command instructing the engine to allocate a new row.
- **VALUES**: A clause containing the exact literal strings, numbers, or functions to be injected. Values can be literals, NULL, scalar subqueries, or substitution variables 7.
- **DEFAULT**: A keyword that instructs the engine to look at the table's DDL definition and inject whatever default value was assigned to that column, bypassing the need to provide a hardcoded value 8.

**Comparison of Insertion Methods:**

Method,Syntax Strategy,Business Use Case
Explicit Declaration,"INSERT INTO tab (col1, col2) VALUES (val1, val2);","Standard UI application inserts. Highly safe 9, 10."
Implicit Declaration,"INSERT INTO tab VALUES (val1, val2, val3);",Fast ad-hoc data entry. Prone to breaking if columns are added later 7.
Subquery Insertion,INSERT INTO tab SELECT * FROM backup_tab;,Mass archiving or cloning data without using the VALUES keyword 7.

**Syntax Breakdown:**

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, DEFAULT, NULL);
```

**Detailed Example:**The HR system needs to hire a new employee named 'JAGRUT' today, but their salary is not yet finalized.

```
INSERT INTO EMP (FIRST_NAME, HIRE_DATE)
VALUES ('JAGRUT', '24-JUN-26');
```

**Execution Logic:**Before placing the data into the database block, the engine rigorously checks constraints. If you attempt to insert NULL or omit a value for a column explicitly defined with a

NOT NULL constraint (like HIRE_DATE), the transaction halts instantly with an **ORA-01400: cannot insert NULL into** error 9, 11.

**Pro-Tips/Best Practices:**

- **Always explicitly declare columns:** Using INSERT INTO table_name VALUES (...) forces you to provide a value for *every single column* in the exact order they were created 7. Always map your columns explicitly (INSERT INTO tab (col1) VALUES (val1)) to insulate your code against future table structure alterations.

## Concept 3: Modifying Data (UPDATE)

**Core Purpose:**The UPDATE command safely alters existing row data without deleting and recreating the record 1, 12. It is capable of updating a single cell, an entire column, or conditionally updating values based on complex mathematical or relational logic 12-14.

**Keyword Breakdown:**

- **UPDATE**: Initiates the modification transaction and places a row-exclusive lock on the targeted data.
- **SET**: Defines which column(s) are being changed and the new value they will hold 12.
- **CASE**: Can be embedded inside the SET clause to apply dynamic, tier-based updates without writing multiple separate UPDATE statements 14.

**Syntax Breakdown:**

```
UPDATE table_name
SET column_1 = new_value,
    column_2 = function(column_2)
WHERE condition;
```

**Detailed Example:**The company declares a dynamic salary increment: 20% for employees hired in Q1 or Q2, and 30% for everyone else 15, 16.

```
UPDATE E1
SET SALARY = CASE
                WHEN TO_CHAR(HIRE_DATE,'Q') IN (1,2) THEN SALARY *
1.2
                ELSE SALARY * 1.3
             END;
```

**Execution Logic:**

1. **Selection:** The engine parses the WHERE clause (if present) to identify target rows 12.
2. **Locking:** It places a lock on those specific rows so no other session can modify them simultaneously.
3. **Modification:** It overwrites the data in the RAM buffer cache and generates Undo data (allowing for ROLLBACK).

**Pro-Tips/Best Practices:**

- **The Missing WHERE Clause:** The WHERE clause is syntactically optional in an UPDATE statement 12. However, omitting it is exceptionally dangerous—it will indiscriminately update **every single row in the entire table** 12. Always double-check your restrictions before execution.

## Concept 4: Removing Data (DELETE)

**Core Purpose:**The DELETE command removes existing rows from a table 1, 17. It can be restricted to target a single specific row, or leverage subqueries to delete rows based on relational data existing in other tables 17, 18.

**Keyword Breakdown:**

- **DELETE FROM**: The DML command that removes rows (the FROM keyword is technically optional in Oracle, DELETE tablename is valid) 17, 19.

**Comparison of Data Removal Commands (DML vs. DDL):**

Feature,DELETE (DML),TRUNCATE (DDL)
Language Category,Data Manipulation Language (DML) 1.,Data Definition Language (DDL) 1.
Can be Rolled Back?,Yes. Generates full undo data 17.,No. Auto-commits immediately 1.
WHERE Clause,Supported. Highly surgical 17.,Not supported. Wipes the entire table.
Performance,Slower (Row-by-row logging).,Instantaneous (Drops the data segment).

**Syntax Breakdown:**

```
DELETE FROM table_name
WHERE condition;
```

**Detailed Example:**The enterprise decides to close the Oxford office and must remove all employees assigned to that location. Because the EMPLOYEES table only contains a DEPARTMENT_ID, we must use a subquery to look up the city 18.

```
DELETE FROM E1
WHERE DEPARTMENT_ID IN (
    SELECT DEPARTMENT_ID
    FROM DEPARTMENTS NATURAL JOIN LOCATIONS
    WHERE CITY = 'Oxford'
);
```

**Execution Logic:**For subquery deletes, the Oracle Engine first executes the inner query block to generate an array of target IDs (e.g., Department IDs for Oxford) 18. It then passes this array to the outer DELETE command, scans the primary table, locks the matching rows, and marks their block headers as empty.

**Pro-Tips/Best Practices:**

- **Never Use DELETE to Wipe a Table:** If you need to completely empty a staging or temporary table containing millions of rows, using DELETE FROM table; will consume massive amounts of CPU and Undo Tablespace memory 17. Always use the DDL command **TRUNCATE TABLE table_name;** 1 instead for instantaneous, resource-free data removal.